UNIVERSITARIA DE COLOMBIA
Ingeniería de Software · Pruebas de Software · 7mo Semestre

TALLER DE CLASE — SESIONES 6 Y 7

Regression · Smoke · Acceptance Testing

Mg. Sergio Alejandro Burbano Mena

abril de 2026

Nombre del estudiante	
Código	
Fecha	
Tiempo total	65 minutos (trabajo en parejas)
Entregable	Archivo .py + archivo .feature + análisis escrito

Contexto del Taller

El Departamento de Sistemas de la Universitaria de Colombia ha desarrollado el módulo de MULTAS AUTOMÁTICAS para la biblioteca. Este módulo calcula multas por libros devueltos tarde y bloquea nuevas reservas a estudiantes con multas pendientes. Acaban de hacer un merge de la rama 'feature/multas' a main y necesitan: 1. Verificar que el build está estable (Smoke Testing) 2. Asegurar que el módulo de préstamos no se rompió (Regression Testing) 3. Validar con el cliente que los criterios de negocio se cumplen (Acceptance Testing)

Parte 1 — Smoke Testing ⏱ 20 minutos

El sistema tiene los siguientes endpoints disponibles:

```
GET  /api/health              → {status: 'ok', db: 'connected'}
POST /api/auth/login          → {access_token: '...', user: {...}}
GET  /api/libros              → [{id, titulo, disponible}, ...]
POST /api/prestamos           → {prestamo_id, libro_id, fecha_devolucion}
POST /api/devoluciones        → {multa: 0.0, estado: 'devuelto'}
GET  /api/multas/{estudiante} → {multa_total, prestamos_vencidos}
```

1.1 Diseña la suite de Smoke Tests

Escribe los 5 smoke tests más críticos para verificar que el nuevo build está estable. Recuerda: deben cubrir lo esencial y ejecutarse en menos de 5 minutos.

```
# tests/smoke/test_biblioteca_smoke.py
```

```
import pytest
import requests

BASE = 'http://localhost:8000/api'

@pytest.mark.smoke
class TestBibliotecaSmoke:

    def test_01_api_disponible(self):
        # TODO: Escribe aquí el test
        pass

    def test_02_login_funciona(self):
        # TODO: Escribe aquí el test
        pass

    def test_03_listar_libros_responde(self):
        # TODO: Escribe aquí el test
        pass

    def test_04_base_de_datos_conectada(self):
        # TODO: Escribe aquí el test
        pass

    def test_05_modulo_multas_responde(self):
        # TODO: Escribe aquí el test
        pass
```

1.2 Preguntas de análisis

1. ¿Por qué es importante que los smoke tests fallen rápido (fail-fast)?

Nos da un análisis rápido, no es probar todo el sistema sino un filtro

2. ¿Qué harías si el smoke test test_01_api_disponible falla? ¿Ejecutarías los demás?

No ejecutaría los demás tests

Parte 2 — Regression Testing ⏱ 25 minutos

El módulo de préstamos tenía las siguientes funciones antes del merge. El equipo de backend informa que el merge de 'feature/multas' modificó la función calcular_disponibilidad() para consultar también las multas. Necesitas asegurarte de que nada se rompió.

Bug histórico documentado

BUG-2024-031: En el sprint anterior, calcular_disponibilidad() devolvía True para libros prestados si el préstamo tenía más de 30 días (suponía devolución automática). Esto fue corregido pero puede reaparecer con el refactor del módulo de multas.

2.1 Escribe los Regression Tests

```
# tests/regression/test_prestamos_regression.py
import pytest
from biblioteca.prestamos import calcular_disponibilidad
from biblioteca.prestamos import registrar_prestamo, registrar_devolucion
from biblioteca.multas    import calcular_multa

@pytest.mark.regression
class TestPrestamosRegression:

    def test_libro_prestado_no_esta_disponible(self):
        """
        REGRESION: Verifica que un libro actualmente prestado
        no aparece como disponible.
        """
        # TODO: Implementa este test
        pass

    def test_libro_disponible_tras_devolucion(self):
        """
        REGRESION: Verifica que el libro queda disponible
        inmediatamente después de la devolución.
        """
        # TODO: Implementa este test
        pass

    def test_bug_2024_031_no_disponible_con_prestamo_antiguo(self):
        """
        REGRESION de BUG-2024-031: Un libro prestado hace más de
        30 días NO debe aparecer como disponible.
        """
        # TODO: Implementa este test — usa datetime para simular
        # un préstamo con fecha de hace 45 días
        pass

    def test_multa_no_afecta_disponibilidad_de_otro_libro(self):
        """
        NUEVO con merge: Tener una multa no debe cambiar la
        disponibilidad de libros que no están prestados.
        """
        # TODO: Implementa este test
        pass

    def test_calculo_multa_tres_dias_retraso(self):
        """
        REGRESION: La multa por 3 días de retraso debe ser
        exactamente 3 * tarifa_diaria (configurada en settings).
        """
        # TODO: Implementa este test
        pass
```

2.2 Preguntas de análisis

3. ¿Por qué se incluyó test_bug_2024_031? ¿Qué tipo de regresión previene?

un error que ya ocurrió y puede reaparecer por cambios previene un regresión con historial

4. ¿Qué tipo de regression testing (completa, parcial, selectiva) usarías para este merge y por qué?

regresión selectiva pruebo directamente lo que cambio

Parte 3 — Acceptance Testing ⏱ 20 minutos

El Product Owner de la biblioteca ha definido las siguientes historias de usuario y criterios de aceptación para el módulo de multas. Tu tarea es escribirlos en formato Gherkin.

Historia de Usuario US-015

COMO estudiante con libros devueltos tarde QUIERO saber el monto exacto de mi multa PARA planificar mi pago antes de hacer nuevas reservas Criterios de aceptación: AC-1: Multa = cantidad de días de retraso × $500 COP por día AC-2: Si la multa es $0, el mensaje debe decir 'Sin multa pendiente' AC-3: Si el estudiante tiene multa > $0, no puede realizar nuevas reservas

3.1 Escribe los escenarios Gherkin

```
# features/multas.feature
# Escribe los escenarios para los 3 criterios de aceptación

Feature: Sistema de multas por devolución tardía
  Como estudiante de la Universitaria de Colombia
  Quiero conocer y gestionar mis multas de biblioteca
  Para poder planificar mis reservas de libros

  # AC-1: Cálculo correcto de la multa
  Scenario: Cálculo de multa por 3 días de retraso
    Given ...
    When  ...
    Then  ...

  # AC-2: Mensaje cuando no hay multa
  Scenario: Estudiante sin multa pendiente
    Given ...
    When  ...
    Then  ...

  # AC-3: Bloqueo por multa pendiente
  Scenario: Estudiante con multa no puede reservar
    Given ...
    When  ...
    Then  ...
    And   ...
```

3.2 Plan UAT — Completa la tabla

ID	Escenario	Pasos	Resultado Esperado	☑/✕
UAT-M01	Cálculo de multa por retraso	1. Tomar un libro 2. Devolverlo con 3 días de retraso 3. Consultar multa	Sistema debe calcular correctamente: 3 × 500 = 1500 COP	
UAT-M02	Estudiante sin multa	1. Tener todas las devoluciones a tiempo 2. Consultar estado de multa	El sistema muestra el mensaje: "Sin multa pendiente"	
UAT-M03	bloqueo por multa pendiente	1. Tener una multa mayor a 0 2. Intentar realizar una reserva	El sistema bloquea la reserva y muestra mensaje de deuda	

3.3 Reflexión final

5. ¿Cuál es la diferencia principal entre que QA apruebe las pruebas y que el cliente/PO las apruebe?

QA aprueba desde lo técnico y funcional ciiente prueba desde el negocio que si funcione com usuario aprueba desde el negocio

6. Si el taller de hoy fuera un proyecto real, ¿en qué orden ejecutarías los 3 tipos de prueba y por qué?

Smoke Testing, Regression Testing,UAT